1. (2 Punkte) Gegeben sind folgende Variablenzuweisungen in Python:

```
a = 7
b = 3.5
c = "Hallo"
d = True
```

Gib für jede Variable den passenden Datentyp an.

Lösung:

```
a: Ganzzahl (int)
b: Gleitkommazahl (float)
c: Zeichenkette (str)
d: Wahrheitswert (bool)
```

2. (3 Punkte) Schreibe ein Python-Programm, das eine Zeichenkette `"42"` in eine Ganzzahl umwandelt, diese um 8 erhöht und das Ergebnis als Zeichenkette wieder ausgibt (Typumwandlung).

Lösung:

```
text = "42"
zahl = int(text)
zahl = zahl + 8
ergebnis = str(zahl)

print(ergebnis)
```

3. (3 Punkte) Erstelle eine leere Liste `a`.
Füge in `a` nacheinander die Zahlen ein: 6, 2, 17
Gib die Liste aus.
Gib die Länge von `a` aus.

Lösung:

```
a = []
a.append(6)
a.append(2)
a.append(17)
print(a)
print(len(a))
```

4. (4 Punkte) Gegeben ist eine Liste `a` mit 6 Elementen, z.B: `a = [4,6,10,2,7,18]`.

a. Der Variable `x` soll das dritten Element der Liste zugewiesen werden. Schreibe 2 Möglichkeiten, wie dies in Python gemacht werden kann.

b. Der Variable `y` soll das letzte Element der Liste zugewiesen werden. Schreibe 3 Möglichkeiten, wie dies in Python gemacht werden kann.

Lösung:

```
a. x = a[2]   oder   x = a[-4]
b. y = a[-1]   oder   y = a[5]   oder   y = a[len(a)-1]
```

5. (3 Punkte) Gegeben ist eine Liste `a = [4,6,10,2,7,18]`.

   Schreibe Python-Anweisungen, die die Liste `a` so verändern, dass sie am Ende wie folgt aussieht: `[2, 6, 12, 2, 7, 18, 5, 9]`.

Lösung:

```
a[0] = 2
a[2] = 12
a.append(5)
a.append(9)
```

6. (5 Punkte) Erkläre anhand eines eigenen Python-Beispiels den Unterschied zwischen den beiden Möglichkeiten, eine Liste mit einer Schleife zu durchlaufen:

Lösung:

```
# Laufvariable i durchläuft die Indizes:
a = [3, 4, 7]
for i in range(len(a)):
    print(i, a[i])

# Laufvariable x durchläuft die Elemente:
a = [3, 4, 7]
for x in a:
    print(x)
```

Unterschied: Bei der Index-Variante steht zusätzlich der Index zur Verfügung (z.B. um a[i] zu verändern oder um auf Nachbarn zuzugreifen), bei der Element-Variante erhält man direkt die Werte, der Code ist kürzer und übersichtlicher.

7. (2 Punkte) Schreibe ein Python-Programm, das mit einer Laufvariablen über den Index jedes Element der Liste `zahlen` um 1 erhöht (die Liste soll anschließend die erhöhten Werte enthalten).

   `zahlen = [4, 7, 2, 9]`

   Erwartete Ausgabe: `[5, 8, 3, 10]`

Lösung:

```
zahlen = [4, 7, 2, 9]

for i in range(len(zahlen)):
    zahlen[i] = zahlen[i] + 1

print(zahlen)
```

8. (4 Punkte) Analysiere den folgenden Python-Code und gib an, welche Ausgabe er erzeugt.

```
a = [2, 4, 9, 12, 1, 5]
b = []
teilsumme = 0
for x in a:
    teilsumme += x
    b.append(teilsumme)
print(b)
```

Lösung:

Bei jedem Durchlauf wird das aktuelle Element x zu teilsumme addiert; der aktuelle Wert von teilsumme wird danach an b angehängt (laufende/kumulierte Summe).

```
x=2:  teilsumme=2  -> b=[2]
x=4:  teilsumme=6  -> b=[2, 6]
x=9:  teilsumme=15 -> b=[2, 6, 15]
x=12: teilsumme=27 -> b=[2, 6, 15, 27]
x=1:  teilsumme=28 -> b=[2, 6, 15, 27, 28]
x=5:  teilsumme=33 -> b=[2, 6, 15, 27, 28, 33]

Ausgabe: [2, 6, 15, 27, 28, 33]
```

9. (4 Punkte) Bestimme, welchen Wert die Variable `index` nach Ausführung des folgenden Codes hat, und begründe deine Antwort.

```
a = [5, 3, 8, 2, 9, 1]
index = -1
for i in range(len(a)):
    if a[i] > 6 and index == -1:
        index = i
print(index)
```

Lösung:

Die Schleife durchsucht die Liste von vorne nach hinten. Sobald ein Element groesser als 6 gefunden wird und index noch -1 ist (also noch kein Treffer gespeichert wurde), wird der Index gemerkt. Da a[2]=8 das erste Element groesser als 6 ist, wird index auf 2 gesetzt und bleibt wegen der Bedingung index == -1 danach unverändert.

Ausgabe: 2

10. (4 Punkte) Der folgende Code berechnet die Summe aller Zahlen einer Liste. Passe den Code so an, dass zusätzlich gezählt wird, wie viele negative Zahlen in der Liste vorkommen.

```
a = [15, 23, 4, 16, 8, 42]
summe = 0
for x in a:
    summe += x
print(summe)
```

Lösung:

```
a = [15, 23, 4, 16, 8, 42]
summe = 0
anzahl_negativ = 0
for x in a:
    summe += x
    if x < 0:
        anzahl_negativ += 1
print(summe)
print(anzahl_negativ)
```

11. (3 Punkte) Schreibe ein Python-Programm, das mit dem Modul `random` eine ganze Zufallszahl zwischen 1 und 6 erzeugt und ausgibt (Simulation eines Würfelwurfs).

Lösung:

```
import random
wurf = random.randint(1, 6)
print("Gewürfelt:", wurf)
```

12. (3 Punkte) Schreibe ein Python-Programm, das 5-mal hintereinander eine Zufallszahl zwischen 1 und 100 erzeugt und alle fünf Zahlen ausgibt.

Lösung:

```
import random

for i in range(5):
    zahl = random.randint(1, 100)
    print(zahl)
```

13. (4 Punkte) Gegeben ist die folgende Python-Funktion und ihr Aufruf:

```
def rechteck_flaeche(breite, hoehe):      # Zeile 1
    flaeche = breite * hoehe              # Zeile 2
    return flaeche                        # Zeile 3

ergebnis = rechteck_flaeche(4, 5)         # Zeile 4
```

Ordne die folgenden Fachbegriffe den passenden Stellen im Code zu: `Name, Kopf, Rumpf, Parameter, Argument, Aufruf, Rückgabewert`

Lösung:

```
Name          : rechteck_flaeche
Kopf          : Zeile 1 (def rechteck_flaeche(breite, hoehe):)
Rumpf         : Zeile 2–3 (der eingerückte Code der Funktion)
Parameter     : breite, hoehe (in der Definition, Zeile 1)
Argument      : 4, 5 (die konkreten Werte beim Aufruf, Zeile 4)
Aufruf        : Zeile 4 (rechteck_flaeche(4, 5))
Rückgabewert  : flaeche, also 20 (der Wert nach return)
```

14. (3 Punkte) Schreibe eine Python-Funktion `ist_gerade(zahl)`, die einen Wahrheitswert zurückgibt, je nachdem ob `zahl` gerade ist. Teste die Funktion mit zwei verschiedenen Zahlen.

Lösung:

```
def ist_gerade(zahl):
    return zahl % 2 == 0

print(ist_gerade(4))
print(ist_gerade(7))
```

15. (4 Punkte) Schreibe eine Python-Funktion `maximum(a, b, c)`, die von drei übergebenen Zahlen die größte zurückgibt, ohne die eingebaute Funktion `max()` zu verwenden.

Lösung:

```
def maximum(a, b, c):
    groesste = a
    if b > groesste:
        groesste = b
    if c > groesste:
        groesste = c
    return groesste

print(maximum(3, 9, 5))
```

16. (3 Punkte) Gegeben ist die folgende Funktion:

```
def mittelwert(a, b):
    return (a + b) / 2
```

Ergänze einen docstring, der beschreibt, welche Parameter die Funktion erwartet und was sie zurückgibt. Gib im docstring außerdem zwei Beispielaufrufe mit ihren Ergebnissen an.

Lösung:

```
def mittelwert(a, b):
    '''
    a, b: zwei Zahlen
    returns: den Mittelwert von a und b

    >>> mittelwert(4, 8)
    6.0

    >>> mittelwert(3, 4)
    3.5
    '''
    return (a + b) / 2
```

17. (4 Punkte) Die Datei `input1.txt` hat folgenden Inhalt:

```
Fahrrad
25
```

Schreibe ein Python-Programm, das die Datei öffnet, in der ersten Zeile den Namen (ohne Zeilenvorschub) in die Variable `artikel` und in der zweiten Zeile den Preis als Ganzzahl in die Variable `preis` einliest. Schließe die Datei anschließend und gib beide Variablen aus.

Lösung:

```
f = open('input1.txt')
artikel = f.readline().strip()
preis = int(f.readline())
f.close()

print(artikel)
print(preis)
```

18. (3 Punkte) Die Datei `input2.txt` enthält den Text `Grüße aus München` (inklusive Umlauten). Schreibe ein Python-Programm, das die Datei mit dem passenden Parameter öffnet, damit die Umlaute korrekt gelesen werden, die Zeile einliest, den Zeilenvorschub entfernt und den Inhalt ausgibt.

Lösung:

```
f = open('input2.txt', encoding='utf-8')
s = f.readline().strip()
f.close()
print(s)
```

19. (4 Punkte) Schreibe ein Python-Programm, das die Variablen `name = "Lena"` und `punkte = 87` anlegt und diese mit `print(..., file=f)` in eine neue Datei `ausgabe.txt` schreibt. Öffne die Datei dazu im Schreibmodus mit `encoding='utf-8'` und schließe sie am Ende.

Erwarteter Inhalt der Datei `ausgabe.txt`:

```
Lena hat 87 Punkte erreicht.
```

Lösung:

```
name = "Lena"
punkte = 87

f = open('ausgabe.txt', encoding='utf-8', mode='w')
print(f'{name} hat {punkte} Punkte erreicht.', file=f)
f.close()
```